

12. DB & Version Control

Attention!

This training is **interview-driven**.

To become a qualified developer, *ChatGPT* and *YouTube* will always be your best friends.

Outline

- Database
- Version Control
- Agile
- SDLC
- Project Management
- CI/CD

DBMS

- **Database Management System (DBMS):** Software that stores, organizes, manages, and retrieves data in databases efficiently and securely
 - Relational DBMS (RDBMS): Uses tables with rows and columns
 - e.g., MySQL, PostgreSQL, Oracle Database, Microsoft SQL Server
 - NoSQL DBMS: Uses flexible models like documents, key-value pairs, graphs
 - e.g., MongoDB, Redis
- Data modeling: the process of analyzing data requirements and defining their relationships
 - One-to-one (1:1)
 - One-to-many (1:N)
 - Many-to-many (N:N)

RDBMS

- A software system that manages databases structured as tables /relations
 - Data is stored in tables, organizes data into rows and columns
 - **Schema-based**: Each table has a predefined schema specifying the columns and their data types
 - Primary keys, Foreign keys
 - SQL language: Uses Structured Query Language (SQL) to insert, update, query, and manage data
 - **ACID** properties: Supports transactions that are Atomic, Consistent, Isolated, and Durable, ensuring reliable and safe data operations
 - Data Integrity: Enforces rules and constraints to maintain accurate data

ACID

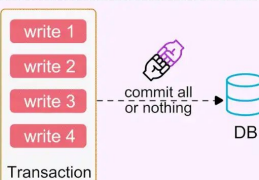
- ACID is a set of properties that guarantee reliable processing of database **transaction**
 - **Atomicity**: Ensures all steps in a transaction succeed, or none at all
 - **Consistency**: Ensures data follows all defined rules and constraints
 - **Isolation**: Ensures transactions are independent, even if they happen together
 - **Durability**: Ensures completed transactions are permanently saved, even after crashes

What does ACID Mean?

ByteByteGo

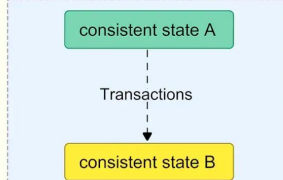
Atomicity

All or nothing



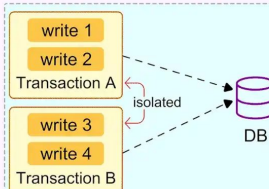
Consistency

Preserving database invariants



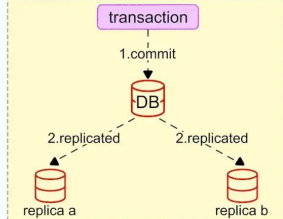
Isolation

Concurrent transactions are isolated from each other



Durability

Data is persisted after transaction is committed even in a system failure



SQL

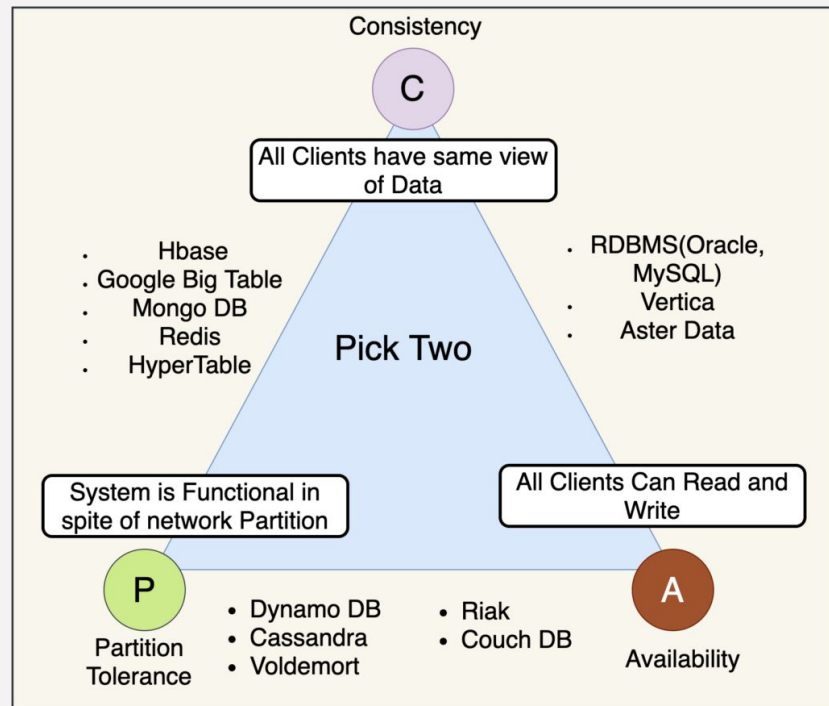
💡 Task	📄 Syntax Example
Select all	<code>SELECT * FROM users;</code>
Select columns	<code>SELECT name, age FROM users;</code>
Filter rows	<code>SELECT * FROM users WHERE age > 18;</code>
Pattern match	<code>SELECT * FROM users WHERE name LIKE 'A%';</code>
Sort results	<code>SELECT * FROM users ORDER BY age DESC;</code>
Limit rows	<code>SELECT * FROM users LIMIT 5;</code>
Insert data	<code>INSERT INTO users (name, age) VALUES ('Tom', 30);</code>
Update data	<code>UPDATE users SET age = 31 WHERE name = 'Tom';</code>
Delete data	<code>DELETE FROM users WHERE age < 18;</code>
Count rows	<code>SELECT COUNT(*) FROM users;</code>
Group results	<code>SELECT gender, COUNT(*) FROM users GROUP BY gender;</code>
Having condition	<code>HAVING COUNT(*) > 1</code>
Join tables	<code>SELECT * FROM orders JOIN users ON orders.user_id = users.id;</code>

NoSQL Databases

- Databases that do not use the traditional table-based relational model
 - Does not use tables or primary/foreign keys, etc
- Key Characteristics
 - Schema-less or flexible schemas: No strict predefined structure
 - Data models:
 - Document databases: Store data as JSON-like documents (**MongoDB**)
 - Key-Value stores: Simple data stored as key-value pairs (Redis, **DynamoDB**)
 - Column-family stores: Store data in columns instead of rows (Cassandra)
 - Graph databases: Store data as nodes and edges (Neo4j)
 - Horizontal scalability
 - High performance
 - Eventual consistency

CAP Theorem

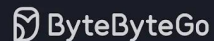
- Consistency (C): Everyone sees the same data at the same time
- Availability (A): Every request gets a response
- Partition Tolerance (P): The system survives network failures
- **CAP Theorem:** A distributed database system can only guarantee 2 out of 3 at the same time
 - CP: MongoDB, Redis
 - AP: DynamoDB, Cassandra



Scalability

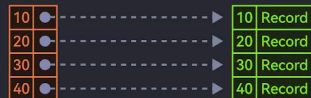
- Scalability in a database refers to its ability to handle increasing amounts of work (like more data, users, or transactions) efficiently, without sacrificing performance
 - Vertical Scalability (Scale Up)
 - Adding more power (CPU, RAM, storage) to an existing machine
 - Horizontal Scalability (Scale Out)
 - Adding more machines (nodes) to a system and distributing the workload
 - Data distribution strategies:
 - **Replication**: Copies data to multiple nodes
 - **Partitioning**: Breaks large tables into smaller parts
 - **Sharding**: Splits data across different servers

Database Scaling Cheatsheet



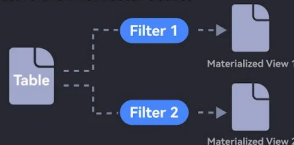
INDEXING

Analyze the query patterns of your application and create the right indexes



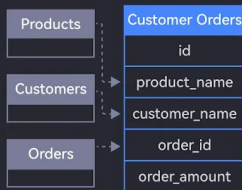
MATERIALIZED VIEWS

Pre-compute complex query results and store them for faster access



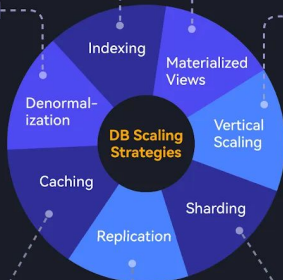
DENORMALIZATION

Reduce complex joins to improve query performance



VERTICAL SCALING

Boost the DB server by adding more CPU, RAM, or storage.



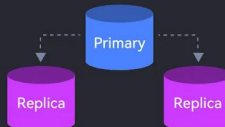
DATABASE CACHING

Store frequently accessed data in a faster storage layer



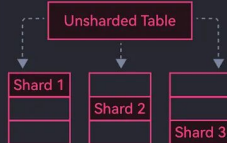
REPLICATION

Create replicas of your primary database on different servers for scaling the reads



SHARDING

Load resources that the page will need before they are needed

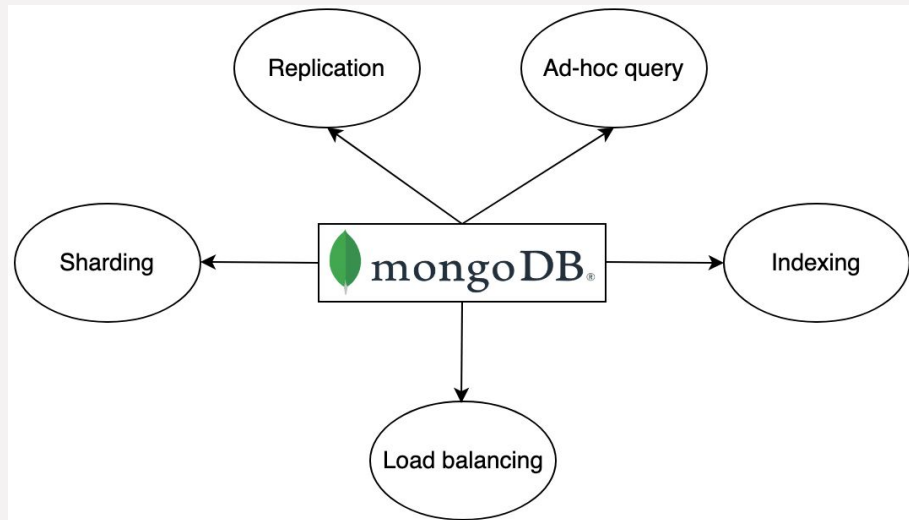


RDBMS vs NoSQL

Feature	Relational Database (RDBMS)	Non-relational Database (NoSQL)
Data Model	Tables with rows and columns	Document, key-value, graph, or column-based models
Schema	Fixed, predefined schema	Flexible or schema-less
Relationships	Supports complex joins and foreign keys	Relationships handled via embedded documents or references (varies by DB)
Query Language	SQL (Structured Query Language)	Varies: JSON-based queries, APIs, or proprietary query languages
Transactions	Full ACID compliance	Limited or tunable ACID, often eventual consistency
Scalability	Primarily vertical (scale-up)	Primarily horizontal (scale-out)
Data Integrity	Strong data integrity with constraints	Weaker constraints, relies on application logic
Use Cases	Structured data, complex queries, enterprise apps	Big data, real-time apps, flexible schema needs
Examples	MySQL, PostgreSQL, Oracle, SQL Server	MongoDB, Cassandra, Redis, Neo4j
Flexibility	Less flexible, requires schema changes for new data types	Highly flexible, easily handles evolving data
Performance	Optimized for complex joins and transactions	Optimized for large volumes, high-speed reads/writes

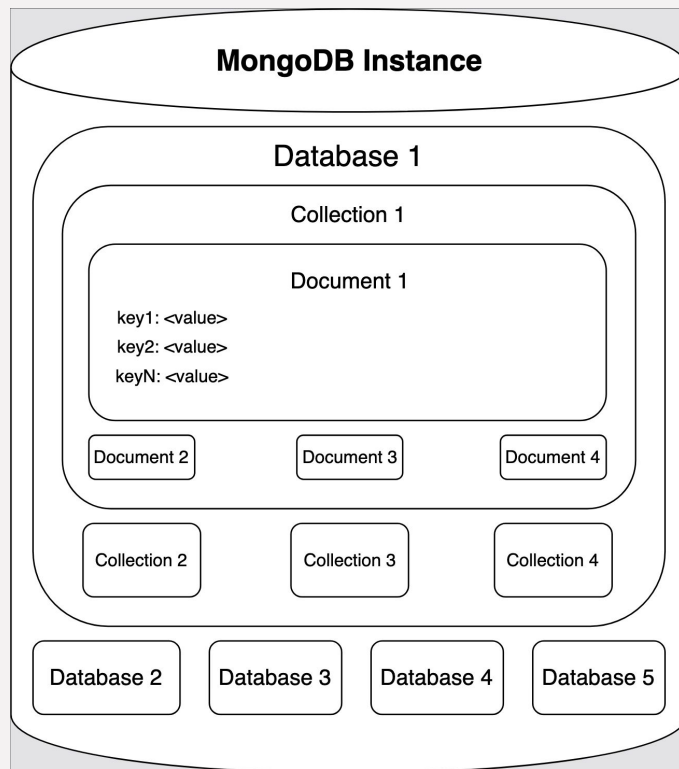
Mongodb

- MongoDB is a NoSQL database that stores data as documents
 - **Schema-less:** Don't need to define the structure the data ahead of time
 - **Document-based:** Each piece of data is stored as a document
 - Highly scalable and fast
 - Naturally supports **JSON**



Document & Collection

- **Document:** A document is a single unit of data
 - Each document is like a row in a SQL database
 - Store and transmit data in **BSON** (Binary JSON) format
- **Collection:** A collection is a group of documents
 - Similar to a table in SQL



MongoDB CRUD

- **Read (Query)**

- `db.collection.find()`
- `db.collection.findOne()`

- **Create (Insert)**

- `db.collection.insertOne()`
- `db.collection.insertMany()`

- **Update**

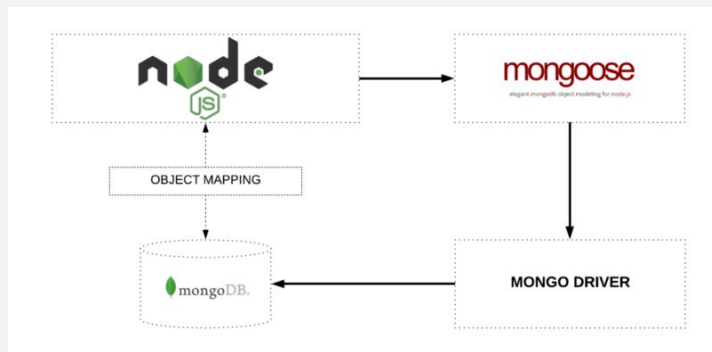
- `db.collection.updateOne()`
- `db.collection.updateMany()`

- **Delete**

- `db.collection.deleteOne()`
- `db.collection.deleteMany()`

Mongoose

- **ODM** (Object Data Modeling/Object Document Mapper): A programming tool that allows developers to interact with document databases like MongoDB using object-oriented code
 - Mongoose: MongoDB connection with Node.js
- Mongoose: A popular ODM library for Node.js
 - Mongoose uses **schemas** and **models** to structure data and interact with MongoDB collections



Schema

- A Schema is like a blueprint or structure definition for documents in a MongoDB collection
 - It defines what fields your documents will have
 - Specifies data types for each field (String, Number, Date, etc.)
 - Allows you to set validation rules, default values, and indexes
 - Controls how data is stored and retrieved
- **SchemaTypes**: String, Number, Date, Buffer, Boolean, Mixed, ObjectId, Array, Decimal128, Map, UUID, Double, Int32
- Validation and error messages
 - Numbers: min and max validator
 - String: enum, match, minLength, maxLength
 - Custom validators: declared by passing a validation function

Model

- Model: Objects created based on Schema definition
 - Instances of a model are documents
 - The model wraps the schema and provides an interface for:
 - Creating documents
 - Querying the database
 - Updating and deleting documents

Concept	What it is	Purpose
Schema	Defines the shape/structure of data	Enforces rules & validation
Model	A class based on the schema	Interacts with MongoDB collection

Version Control

- A system that records changes to files over time so that you can track history, collaborate with others, and revert to previous versions when needed
- Distributed (DVCS): Everyone has a full copy of the repo, including history
 - **Git** is a distributed version control system used to track code changes, collaborate with others, and manage code versions

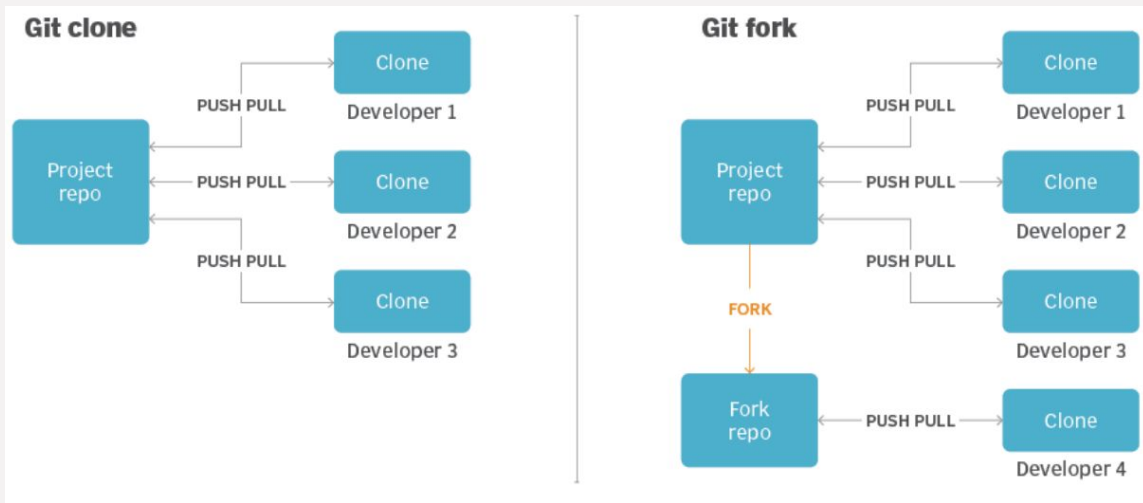
Feature	Centralized Version Control (CVC)	Distributed Version Control (DVC)
Structure	Everyone connects to a central server	Everyone has a full copy of the repository
Examples	SVN, CVS, Perforce	Git, Mercurial, Bazaar
Storage	Central server stores full history; local has only working copy	Local and remote both store full history

Key Concepts

概念	英文术语	说明
仓库	Repository	存储项目代码的地方，可以是本地或远程。
提交	Commit	一次保存当前更改的记录。
分支	Branch	一个独立的开发线，默认是 <code>main</code> 或 <code>master</code> 。
合并	Merge	把一个分支的更改合并到另一个分支。
克隆	Clone	复制一个远程仓库到本地。
暂存区	Staging Area	预提交文件的地方，通过 <code>git add</code> 添加。
HEAD	HEAD	指向当前分支的最后一次提交。

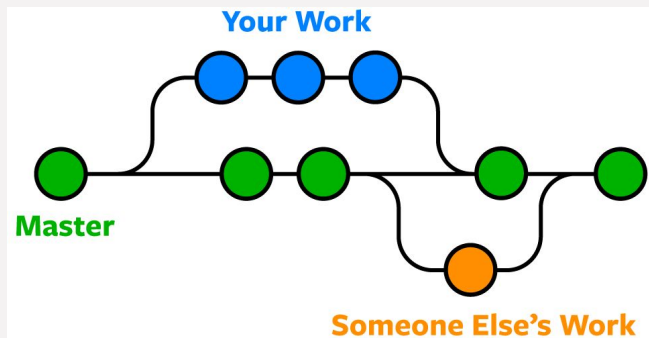
Fork vs Clone

- Fork: Creates a copy of someone else's repo under your GitHub account
 - Used when you want to contribute to someone else's project
 - Must fetch & rebase updates from the original repo onto yours
- Clone: Downloads a repo to your computer
 - Used when you want to work on a repo locally



Branch

- Branch: a version of the repo that is isolated from the main version
 - Mainly used by developers to work on features or test implementation ideas independently
 - Often merged into the main branch
- git branch: view all existing branches and the one you're currently on
- git branch branchName: creates a new branch named "branchName"
- -m main: renames the master branch to "main"
- -d branchName: delete a branch
- git switch branchName: switches to the branch, allowing you to make changes only on that branch



Pull request

- Pull request (**PR**): asking for your local repo's branch to be “pulled” and combined into the main repo's branch
 - **PR/Code Review**
- Status: open, closed
- Other developers can review changes, add additional commits or comments, then eventually merge or rebase them onto the main branch

Basic Git Commands

- `git init`: create a new local repository
- `git clone <repo_url>`
- `git checkout -b feature/new-feature`
- Developing...
- `git add .`
- `git commit -m "Add new feature"`
- `git push origin feature/new-feature`
- Create Pull Request
- `git pull`: download changes from remote repository
- **.gitignore**: a file that specifies which files & directories should be excluded when uploading/pushing to GitHub

Common Git commands

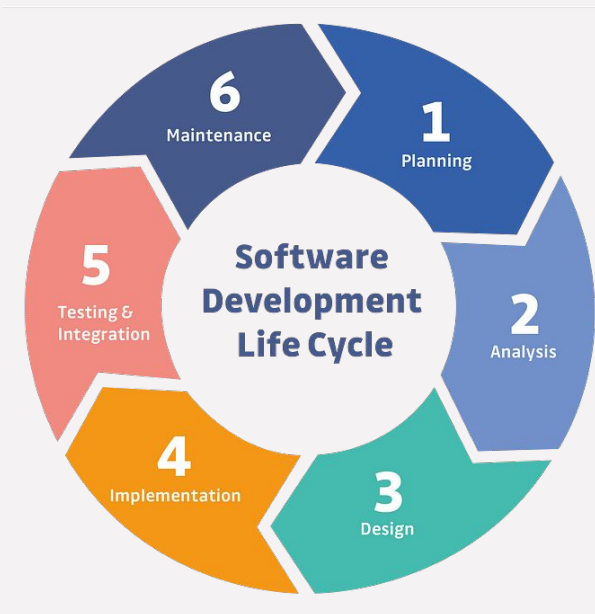
Category	Command	Purpose
Stage changes	<code>git add</code>	Move changes to staging area
Commit	<code>git commit</code>	Save changes to local repo
Push	<code>git push</code>	Send changes to remote
Pull	<code>git pull</code>	Get + merge changes from remote
Stash	<code>git stash</code>	Save changes temporarily
Inspect	<code>git status</code> , <code>log</code> , <code>diff</code>	Check what's going on
Undo	<code>git restore</code> , <code>reset</code> , <code>revert</code>	Revert files or commits
Branching	<code>git branch</code> , <code>checkout</code> , <code>switch</code>	Manage branches
Merging	<code>git merge</code> , <code>rebase</code> , <code>squash</code>	Combine or clean up history

Agile

- Agile is a **project management** that emphasizes flexibility, collaboration, and customer-focused development
 - Team: builds the product
 - Customer or Product Owner: decides what's important
 - **Scrum** Master (if using Scrum): helps the team follow the process
- Agile Cycle
 - Plan → Design → Build → Test → Deliver → Review → Repeat
 - Each cycle is often called an iteration or **sprint** (in Scrum)

SDLC

- Software Development Life Cycle (SDLC): a process of designing, developing, and testing software in order to minimize the cost and maximize the quality of development
- **Typical environments**
 - **Development:** the local machine, where developers write code, fix bugs, write unit tests
 - **Testing/QA:** frequently executes automated tests in parallel
 - **Staging:** A production-like environment for final testing
 - **Production:** Where real users interact with your app

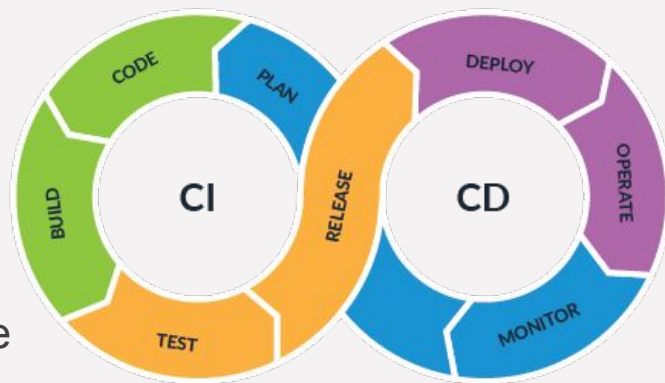


Project Management

- **Scrum:** An **agile** project management framework followed by Scrum teams, it helps teams work together more effectively by using short, focused work cycles called sprints
 - **Scrum Events**
 - Sprint Planning: Decide what to work on in the upcoming sprint
 - Daily Scrum (Stand-up): 15-minute daily sync to track progress and issues
 - Sprint Review: Team demo and get feedback on the work done
 - **Sprints:** a period of time (2~4 weeks) with well-defined goals & deliverables
 - Popular Scrum **Tools:** Jira (by Atlassian), Trello, Monday.com
- Key Roles & Responsibilities in a Software Development Team
 - Team/Tech Lead, Project Manager (PM), Developers (frontend/backend/full stack), UI/UX Design, Scrum Master, QA Team

CI/CD

- **Continuous Integration & Continuous Delivery (CI/CD):** Source code changes enter a pipeline to ensure that they meet all standards before delivery
 - You push code
 - Tests automatically run
 - App builds and deploys to staging (or production)
 - You get notified of success/failure
- **CI:** Automatically build and test code after every change
- **CD**
 - CD (Delivery): Automatically prepare code for release
 - CD (Deployment): Automatically release code to users



Popular CI/CD Tools

Tool	Role
GitHub Actions	Native CI/CD in GitHub
GitLab CI/CD	Integrated with GitLab
Jenkins	Highly customizable CI/CD platform
CircleCI, Travis CI	Cloud-based CI tools
AWS CodePipeline, Azure DevOps, Bitbucket Pipelines	Platform-native CI/CD services

Homework - Short Answer Questions

1. What is a relational database?
2. What is a primary key and a foreign key?
3. What is a non-relational database?
4. What is the difference between vertical and horizontal scaling?
5. When would you choose a RDBMS vs NoSQL database?
6. What is ACID
7. What is CAP?
8. What is Collection in MongoDB?
9. What is Mongoose? explain Schema and Model
10. What is CI/CD?

Homework - Coding Questions

Everyone should use GitHub to manage your individual full-stack project, including:

Creating a new repository. Try to Use basic Git commands such as

- init, commit, push, and pull, Switch branch
- set up **.gitignore**...

1. 初始化 git

```
git init
```

2. 关联远程仓库

```
git remote add origin https://github.com/用户名/仓库名.git
```

3. 添加文件

```
git add .
```

4. 提交

```
git commit -m "init"
```

5. 推送

```
git branch -M main
```

```
git push -u origin main
```